Sprint 6 — Observer Pattern

Class 1

Lesson 1 — Why Observer Exists

Learning Objective

By the end of this lesson, we should be able to answer:

- What problem does Observer Pattern solve?
- Why direct communication creates tight coupling?
- What are Publisher and Observer?
- When should Observer Pattern be used?

Start With a Real Enterprise Problem

Imagine we are building **Room Booking Platform**.

A new owner user registers.

What should happen?

User registers
|
Save User
|
Send Welcome Email
Send SMS
Create Owner Profile
Create Audit Log
Update Analytics
Notify Admin
Award Welcome Points

Question:

Who should be responsible for all these actions?

Most beginners answer:

UserService

Let's see.

Bad Design

```
@Service
public class UserService {

    public void register(CreateUserRequest request) {

        validate(request);

        User user = userRepository.save(request);

        emailService.sendWelcomeEmail(user);

        smsService.sendSms(user);

        ownerProfileService.create(user);

        auditService.log(user);

        analyticsService.track(user);

        notificationService.notifyAdmin(user);

        loyaltyService.giveWelcomePoints(user);
    }

}
```

How many services does UserService know?

EmailService

SMSService

AuditService

AnalyticsService

OwnerProfileService

NotificationService

LoyaltyService

Seven!

What Happens Tomorrow?

Business says:

Add Telegram Notification.

Modify UserService.

Next week:

Add Kafka Event.

Modify UserService.

Next month:

Add Slack Notification.

Modify UserService again.

Eventually

UserService

1500 lines

Which SOLID Principle Is Violated?

Open/Closed Principle

Every new feature => Modify UserService

Why Observer Exists

Instead of this

UserService -> Email -> SMS -> Analytics -> Audit

Observer says

UserService -> Publish Event -> Observers decide whether they care

Now UserService knows only EventPublisher

Nothing else.

Lesson 2 — Understanding the Observer Pattern

Observer Pattern has four participants.

Subject (Publisher)

Something happened.

Example

-User Registered
-Order Created
-Payment Completed
-Room Approved

Observer

Objects interested in that event.

-Example

-Email
-SMS
-Analytics
-Audit
-Notification

Concrete Observer

Actual implementations.

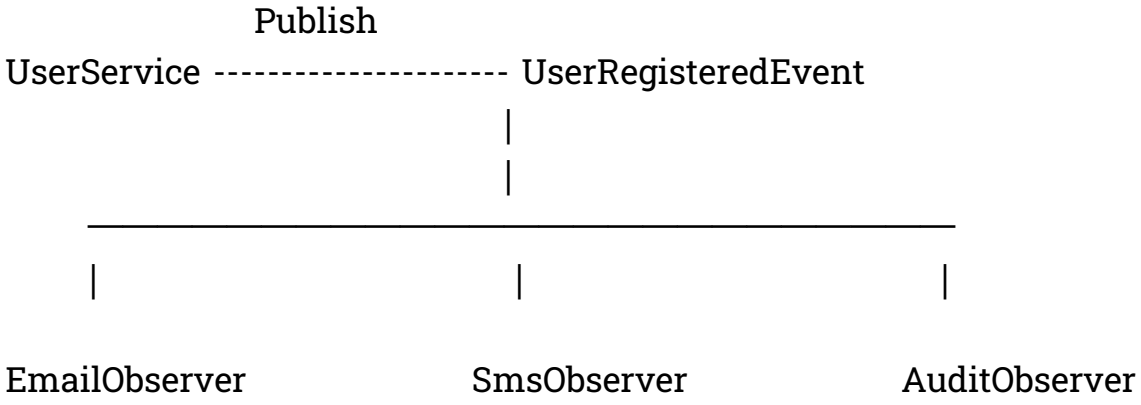
-WelcomeEmailObserver
-SmsObserver
-AuditObserver
-AnalyticsObserver

Event

The information being shared.

```
public class UserRegisteredEvent {

    private final User user;

    public UserRegisteredEvent(User user) {
        this.user = user;
    }

    public User getUser() {
        return user;
    }

}
```

Observer Diagram



Lesson 3 — Implement Observer (Plain Java)

Step 1

Observer Interface

```
public interface UserObserver {

    void onUserRegistered(User user);

}
```

Step 2

Concrete Observer

Email

```
public class WelcomeEmailObserver
        implements UserObserver {

    @Override
    public void onUserRegistered(User user) {
        System.out.println(
                "Send welcome email to "
                        + user.getEmail());
    }

}
```

SMS

```
public class SmsObserver
        implements UserObserver {

    @Override
    public void onUserRegistered(User user) {
        System.out.println(
                "Send SMS to "
```

```
                    + user.getPhone());
    }

}
```

Audit

```
public class AuditObserver
        implements UserObserver {

    @Override
    public void onUserRegistered(User user) {
        System.out.println(
                "Audit user "
                        + user.getUsername());
    }

}
```

Lesson 4 — Publisher

```
import java.util.ArrayList;
import java.util.List;

public class UserEventPublisher {

    private final List<UserObserver> observers =
            new ArrayList<>();

    public void register(UserObserver observer) {
        observers.add(observer);
    }
```

```
    public void publish(User user) {

        for (UserObserver observer : observers) {
            observer.onUserRegistered(user);
        }

    }

}
```

Lesson 5 — UserService

```
public class UserService {

    private final UserEventPublisher publisher;

    public UserService(UserEventPublisher publisher) {
        this.publisher = publisher;
    }

    public void register(User user) {

        System.out.println("Save User");

        publisher.publish(user);

    }

}
```

Demo

```
public class ObserverDemo {
```

```
    public static void main(String[] args) {

        UserEventPublisher publisher = new UserEventPublisher();

        publisher.register(new WelcomeEmailObserver());

        publisher.register(new SmsObserver());

        publisher.register(new AuditObserver());

        UserService service = new UserService(publisher);

        User user = new User(
                "piseth",
                "piseth@gmail.com",
                "012345678");

        service.register(user);

    }

}
```

Output

```
Save User

Send welcome email to piseth@gmail.com

Send SMS to 012345678

Audit user piseth
```

Test understanding Moment

How many observers can we add?

Answer

Unlimited.

Tomorrow

Add

TelegramObserver

SlackObserver

AnalyticsObserver

KafkaObserver

DashboardObserver

UserService

No change.

Lesson 6 — SOLID Connection

SRP

UserService : Responsible only for Register User

Email: Responsible only for Send Email

Audit: Responsible only for Write Audit

OCP

Need new observer?

Create new class.

No modification.

DIP

UserService depends on UserEventPublisher

Not EmailService, SMSService, AnalyticsService

So, Much cleaner.

Common Mistakes

Mistake 1

Publisher doing business logic.

Publisher should only notify.

Mistake 2

Observers calling each other.

Bad:

Email -> SMS -> Audit

Observers should be independent.

Mistake 3

One observer knowing another observer.

Observer should only know Event.

Homework

Build Order System

When Order Created notify:

-InventoryObserver
-EmailObserver
-AuditObserver
-ShippingObserver

Tasks

1. Create OrderCreatedEvent
2. Create OrderObserver
3. Create 4 concrete observers.
4. Create OrderPublisher.
5. Demonstrate adding a fifth observer without changing OrderService.

Interview Questions

1. What problem does Observer Pattern solve?
2. Why is Observer considered a behavioral pattern?
3. What is the difference between Publisher and Observer?
4. Which SOLID principles does Observer support?
5. Can one event have multiple observers?
6. What happens if no observer is registered?
7. Give three real-world examples of Observer Pattern.
8. How does Observer reduce coupling?